***PRACTICAL – 1***

# *Implementation and Time Analysis of Sorting Algorithms*

***Objective:***

The objective of this practical is to implement and compare the performance of ***Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort***. We will analyze the time complexity and execution time of each algorithm to understand their efficiency and identify the most suitable algorithm for different applications.

## *1. Bubble Sort*
## *Algorithm:*

1. Start

2.For i= 0 to n-2

3. For j=0 to n-i-2

4. IfA[j]>A[j+1]

5. SwapA[j]and A[j + 1]

6. End If

7. End For

8. End For

9. Stop

## *Code:*

```
import time

# Bubble Sort Function
def bubble_sort(arr, n):
    for i in range(n - 1):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swapping occurs, array is already sorted
        if not swapped:
            break


# Take input from the user
n = int(input("Enter the number of elements: "))
```

```
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Measure execution time
start_time = time.perf_counter()

bubble_sort(arr, n)

end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f}
seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

***OUTPUT:***

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Users\penta\AppData\Loca
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60960' '--
74
657
Sorted array: 1 3 3 4 28 74 94 657 893

Time Complexity:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)
Enrollment No: 92460118708
```

***Time Complexity:***

| *Case* | *Time Complexity* | *Description* |
|---|---|---|
| *Best Case* | *O(n)* | *The array is already sorted. With the optimized version (using a swapped flag), only one pass is required.* |
| *Average Case* | *$O(n^2)$* | *The array is in random order, requiring multiple comparisons and swaps.* |
| *Worst Case* | *$O(n^2)$* | *The array is sorted in reverse order, requiring the maximum number of comparisons and swaps.* |

## *Space Complexity:*

| *Space Complexity* | *Description* |
|---|---|
| *O(1)* | *Uses only a temporary variable for swapping.* |

## *2. Selection Sort*

## *Algorithm:*

1. Start
2. For i = 0 to n - 2
3. min = i
4. For j = i + 1 to n - 1
5. If A[j] < A[min]
6. min = j
7. End If
8. End For
9. Swap A[i] and A[min]
10. End For
11. Stop

## *Code:*

```
import time

# Selection Sort Function
def selection_sort(arr):
    n = len(arr)
```

```
    for i in range(n):
        min_index = i

        # Find the minimum element in the unsorted part
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum element with the first unsorted element
        arr[i], arr[min_index] = arr[min_index], arr[i]


# Take input from the user
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Measure execution time
start_time = time.perf_counter()

selection_sort(arr)

end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

***OUTPUT:***

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\deb
25
Sorted array: 25 35 78 98

Time Complexity:
Best Case: O(n^2)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)

Enrollment Number: 92460118708
```

***Time Complexity:***

| *Case* | *Time Complexity* | *Description* |
|---|---|---|
| *Best Case* | *$O(n^2)$* | *Even if the array is already sorted, the algorithm scans the remaining unsorted elements to find the minimum in every pass.* |
| *Average Case* | *$O(n^2)$* | *The array is in random order, requiring the same number of comparisons in each pass.* |
| *Worst Case* | *$O(n^2)$* | *The array is in reverse order, but the algorithm still performs the same number of comparisons and selects the minimum element in each pass.* |

## *Space Complexity:*

| *Space Complexity* | *Description* |
|---|---|
| *O(1)* | *Sorts in place using constant extra memory.* |

## *3. Insertion Sort*

### *Algorithm:*

1. Start
2. For i = 1 to n - 1
3. key = A[i]
4. j = i - 1
5. While j >= 0 AND A[j] > key
6. A[j + 1] = A[j]
7. j = j - 1
8. End While
9. A[j + 1] = key
10. End For
11. Stop

### *Code:*

```
import time

# Insertion Sort Function
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1
        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
insertion_sort(arr)
end_time = time.perf_counter()
# Display the sorted array
print("\nSorted Array:", arr)
```

```
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

***OUTPUT:***

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Use
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
62
12
14
Sorted array: 12 14 45 62 89
Time Complexity:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)
Enrollment No: 92460118708
```

***Time Complexity:***

| *Case* | *Time Complexity* | *Description* |
|---|---|---|
| *Best Case* | *O(n)* | The array is already sorted, so each element is compared only once without any shifting. |
| *Average Case* | *O(n²)* | The array is randomly ordered, requiring comparisons and shifts for inserting elements into their correct positions. |
| *Worst Case* | *O(n²)* | The array is sorted in reverse order, requiring the maximum number of comparisons and shifts for every element. |

## *Space Complexity:*

| *Space Complexity* | *Description* |
|---|---|
| *O(1)* | *Uses one extra variable for inserting elements.* |

## 4. Merge Sort :

### *Algorithm:*

1. If left < right
2. mid = (left + right) / 2
3. MergeSort(A, left, mid)
4. Merge Sort(A, mid + 1, right)
5. Merge(A, left, mid, right)
6. End If

### *Code:*

```
import time
# Merge Sort Function
def merge_sort(arr):
    if len(arr) > 1:
        # Find the middle of the array
        mid = len(arr) // 2
        # Divide the array into two halves
        left = arr[:mid]
        right = arr[mid:]
        # Recursively sort both halves
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        # Merge the sorted halves
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        # Copy remaining elements of left half
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
```

```
        # Copy remaining elements of right half
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
merge_sort(arr)
end_time = time.perf_counter()
# Display the sorted array
print("\nSorted Array:", arr)
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

***OUTPUT:***

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Users\pent
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
37
Sorted array: 9 35 37 74 89

Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n log n)
Space Complexity: O(n)

Enrollment Number:92460118708
```

***Time Complexity:***

| *Case* | *Time Complexity* | *Description* |
|---|---|---|
| *Best Case* | *O(n log n)* | *The array is recursively divided and merged efficiently regardless of its initial order.* |
| *Average Case* | *O(n log n)* | *Randomly ordered elements are divided into subarrays and merged in sorted order.* |
| *Worst Case* | *O(n log n)* | *Even in the worst arrangement, the algorithm performs the same divide-and-conquer operations.* |

## *Space Complexity:*

| *Space Complexity* | *Description* |
|---|---|
| *O(n)* | *Uses an additional array during merging.* |

## *5. Quick Sort:*

## *Algorithm:*

```
if low < high:
    pivot = Partition(arr, low, high)
    QuickSort(arr, low, pivot - 1)
    QuickSort(arr, pivot + 1, high)
return arr
```

## *Code:*

```
import time
# Partition Function
def partition(arr, low, high):
    pivot = arr[high]      # Choose last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
# Quick Sort Function
def quick_sort(arr, low, high):
    if low < high:
```

```
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
quick_sort(arr, 0, n - 1)
end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

***OUTPUT:***

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\U
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugp
99
Sorted array: 46 87 94 99

Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n^2)
Space Complexity: O(log n) average

Enrollment Number: 92460118708
```

***Time Complexity:***

| *Case* | *Time Complexity* | *Description* |
|---|---|---|
| *Best Case* | *O(n log n)* | *The pivot divides the array into two nearly equal halves at every recursive step.* |
| *Average Case* | *O(n log n)* | *On average, the pivot creates reasonably balanced partitions, resulting in efficient sorting.* |
| *Worst Case* | *O(n²)* | *The pivot is always the smallest or largest element, creating highly unbalanced partitions (e.g., an already sorted array with a poor pivot choice).* |

## *Space Complexity:*

| *Space Complexity* | *Description* |
|---|---|
| *O(log n) (Average)* | Uses recursion stack during sorting. |
| *O(n) (Worst Case)* | Occurs when partitions are highly unbalanced. |